



Programmation Python

Filière LSI1



Chapitre 3:

Les fonctions

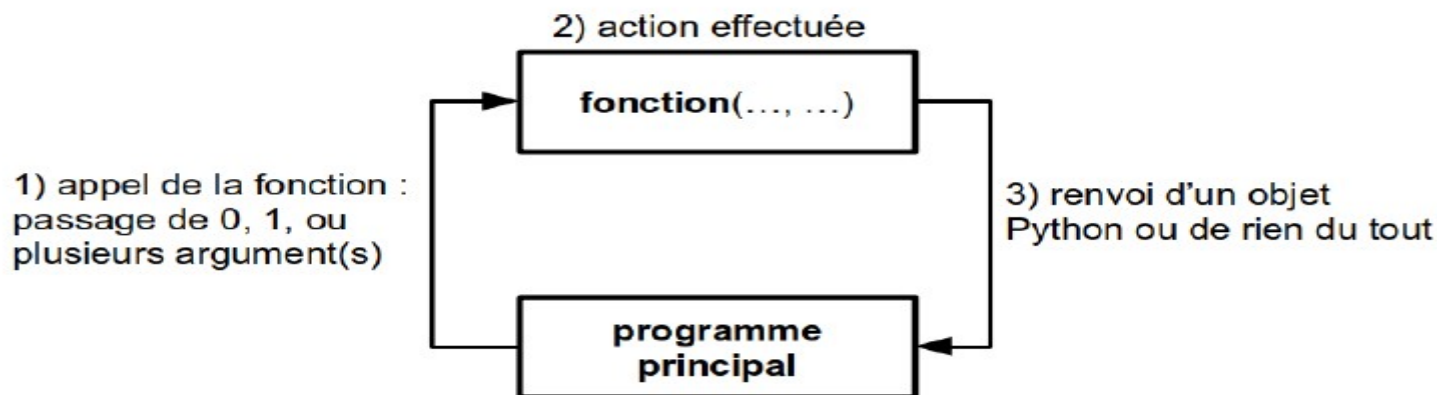
2020-2021 - SEMESTRE 2

nassima.medimegh@fsm.rnu.tn



1. Définition

- Le langage Python possède déjà des fonctions prédéfinies comme `print()` pour afficher du texte ou une variable, `input()` pour lire une saisie clavier... Mais il offre à l'utilisateur la possibilité de créer ses propres fonctions



python 2. Syntaxe

- Pour définir une fonction, Python utilise le mot-clé **def** :

```
def Nom_De_La_Fonction(liste de paramètres):  
    ...  
    bloc d'instructions
```

```
>>>def somme(a):  
    a=a+1  
    print (a)
```



3. Renvoi de résultats

- Un énorme avantage en Python est que les fonctions sont capables de renvoyer plusieurs objets à la fois
- Si on souhaite que la fonction renvoie quelque chose, il faut utiliser le mot-clé **return**. Par exemple :

```
>>> def carre(x):  
    ... return x**2  
    ...  
>>> print(carre(2))  
4
```



4. Vraies fonctions et procédures

- Une « vraie » fonction doit en effet *renvoyer une valeur* lorsqu'elle se termine.
- Exemple:

y = sin(a)

On comprend que dans cette expression, la fonction **sin()** renvoie une valeur qui est directement affectée à la variable y.



5. Passage d'arguments

- Le nombre d'arguments que l'on peut passer à une fonction est **variable**.
- Comme Python est connu comme étant un langage au « **typage dynamique** », vous n'êtes pas obligé de préciser le type des arguments



5. Passage d'arguments

1. Arguments positionnels

- Lorsqu'on définit une fonction `def fct(x, y):` les arguments `x` et `y` sont appelés **arguments positionnels**.
- Il est strictement obligatoire de les préciser lors de l'appel de la fonction.
- Il est nécessaire de respecter le même ordre lors de l'appel que dans la définition de la fonction.



5. Passage d'arguments

Exemple

```
>>> def multiplication(x, y):  
    return x*y  
  
>>> multiplication(3, 2)  
6  
>>> multiplication(3)  
Traceback (most recent call last):  
  File "<pyshell#8>", line 1, in <module>  
    multiplication(3)  
TypeError: multiplication() missing 1 required positional argument: 'y'
```




5. Passage d'arguments

2. Arguments par mot-clé

- Un argument défini avec **def fct(arg=val):** est appelé **argument par mot-clé**.
- Le passage d'un tel argument lors de l'appel de la fonction est facultatif.

```
>>> def fct(x=0, y=0, z=0):  
...     return x, y, z  
...  
>>> fct()  
(0, 0, 0)  
>>> fct(10)  
(10, 0, 0)  
>>> fct(10, 8)  
(10, 8, 0)  
>>> fct(10, 8, 3)  
(10, 8, 3)
```



5. Passage d'arguments

2. Arguments par mot-clé

- Si les arguments par mot-clé ont reçu chacun une valeur par défaut, on peut faire appel à la fonction en fournissant les arguments correspondants **dans n'importe quel ordre, à la condition de désigner nommément les paramètres correspondants.**



5. Passage d'arguments

2. Arguments par mot-clé

```
def Exemple(a=1, b="Programmation", c="Python"):  
    print(a,b,c)
```

```
Exemple(b="cours", a=2, c="JAVA")
```

```
#2 cours JAVA
```

```
Exemple()
```

```
#1 Programmation Python
```



6. Variables locales et globales

- Une variable est dite **locale** lorsqu'elle est créée dans une fonction. Elle n'existera et ne sera visible que lors de l'exécution de la fonction.
- Une variable est dite **globale** lorsqu'elle est créée dans le programme principal. Elle sera visible partout dans le programme.



6. Variables locales et globales

```
>>> def exemple():  
    p=20 #p locale variable  
    print(p,q)  
  
>>> p,q=100,200 #global variable  
>>> exemple()  
20 200  
>>> print(p,q)  
100 200
```



6. Variables locales et globales

Modifier une variable globale

- Pour définir une fonction qui soit capable de modifier une variable globale, il suffit d'utiliser l'instruction **global**.



6. Variables locales et globales

Modifier une variable globale

- Exemple :

```
>>>def test()
    global a
    a=a+1
>>> a=15
>>>test()
>>>print(a)
16
```

- La variable **a** utilisée à l'intérieur de la fonction `test()` est non seulement accessible, mais également modifiable, parce qu'elle est signalée explicitement comme étant une variable qu'il faut traiter globalement



7. Fonction lambda

Fonction lambda:

- En Python, la fonction *lambda* est une fonction anonyme (à laquelle, on n'a pas donné de nom), et qu'on peut l'appliquer dans une expression.
- La syntaxe est : *lambda paramètres: expression*
- La fonction *lambda* est réservée à des situations relativement simples. Sa définition doit tenir sur une seule ligne, et elle ne peut pas contenir d'instructions composées (pas d'affectation, pas de boucle, etc.). Elle consiste donc essentiellement en la définition d'une expression calculée en fonction des paramètres qui lui sont passés.



7. Fonction lambda

Fonction lambda:

- Les deux définitions suivantes de la fonction f sont équivalentes :

```
>>>def f(x, y, z)
    return 100*x+10*y+z
>>>f(1, 2, 3)
123
```

```
>>> f = lambda x, y, z: 100*x+10*y+z
>>> f(1,2,3)
123
```



8. La notion d'exception

- Quand une erreur se produit dans un script, elle provoque l'arrêt du programme et l'affichage d'un message d'erreur.
- Pour éviter cette interruption brutale, on peut appliquer un traitement spécifique. Plus généralement, on peut traiter une situation exceptionnelle dont on ne sait pas forcément où et quand elle va survenir à l'intérieur d'un bloc donné. Plutôt que de parler d'erreur, on emploie donc le terme « **exception** » Et prévoir une réaction adaptée à une exception.

try: ... except :



8. La notion d'exception

try :

bloc 1_dans_lequel_une_exception_peut_survenir

except :

bloc2_de_rattrapage_de_toutes_les_exeptions

- le bloc 2 est parcouru si une exception (quelle qu'elle soit) est rencontrée dans le bloc 1 (cette exception provoque l'arrêt du bloc 1 et le passage immédiat au bloc 2) :

try :

n = int(input("entrer un entier:"))

except :

print("Ce n'est pas un entier valide ")

entrer un entier:abc

Ce nest pas un entier valide



8. La notion d'exception

- Python possède beaucoup d'exceptions prédéfinies, voici quelques-unes

IndexError	Se produit si on cherche à accéder à un élément d'une liste en dehors des limites de celle-ci
NameError	Se produit quand on évoque une variable (local ou globale) dont le nom n'existe pas
SyntaxError	Se produit quand Python échoue à lire une expression dont la syntaxe est erronée.
TypeError	Se produit quand une opération est appliquée à un objet qui n'est pas du type adéquat.
ZeroDivisionError	Se produit quand un calcul arithmétique conduit à une division par 0



8. La notion d'exception

Exemple

```
def division(a, b):  
    try:  
        d = a//b  
    except ZeroDivisionError:  
        print("Division par zero ! ")  
    else:  
        print("Resultat : ", d)
```

Résumé : structure d'un programme Python type

```
# -*- coding:Utf8 -*-
#####
# Programme Python type
# auteur : G.Swinnen, Liège, 2009
# licence : GPL
#####

#####
# Importation de fonctions externes :
from math import sqrt

#####
# Définition locale de fonctions :
def occurrences(car, ch):
    "Cette fonction renvoie le \
    nombre de caractères <car> \
    contenus dans la chaîne <ch>"
    nc = 0
    i = 0
    while i < len(ch):
        if ch[i] == car:
            nc = nc + 1
        i = i + 1
    return nc

#####
# Corps principal du programme :
print("Veuillez entrer un nombre :")
nbr = eval(input())
print("Veuillez entrer une phrase :")
phr = input()
print("Entrez le caractère à compter :")
cch = input()
no = occurrences(cch, phr)
rc = sqrt(nbr**3)
print("La racine carrée du cube", end=' ')
print("du nombre fourni vaut", end=' ')
print(rc)
```

Un programme Python contient en général les blocs suivants, dans l'ordre :

- Quelques instructions d'initialisation (importation de fonctions et/ou de classes, définition éventuelle de variables globales).
- Les définitions locales de fonctions et/ou de classes.
- Le corps principal du programme.

Le programme peut utiliser un nombre quelconque de fonctions, lesquelles sont définies localement ou importées depuis des modules externes. Vous pouvez vous-même définir de tels modules.

La définition d'une fonction comporte souvent une liste de PARAMÈTRES. Ce sont toujours des VARIABLES, qui recevront leur valeur lorsque la fonction sera appelée.

Une boucle de répétition de type 'while' doit toujours inclure au moins quatre éléments :

- l'initialisation d'une variable 'compteur' ;
- l'instruction while proprement dite, dans laquelle on exprime la condition de répétition des instructions qui suivent ;
- le bloc d'instructions à répéter ;
- une instruction d'incrément du compteur.

La fonction "renvoie" toujours une valeur bien déterminée au programme appelant. Si l'instruction 'return' n'est pas utilisée, ou si elle est utilisée sans argument, la fonction renvoie un objet vide : 'None'.

Le programme qui fait appel à une fonction lui transmet d'habitude une série d'ARGUMENTS, lesquels peuvent être des valeurs, des variables, ou même des expressions.

Exercice 1: Créez une fonction `est_premier()` qui prend comme argument un nombre entier positif `n` (supérieur à 2) et qui renvoie le booléen `True` si `n` est premier et `False` si `n` n'est pas premier.

- 1- Saisir un nombre entier positif `n` (supérieur à 2) en utilisant les exceptions.
- 2- Créer la fonction `est_premier()`
- 3- Déterminer tous les nombres premiers de 2 à 100 et les enregistrer dans un dictionnaire.